PAYMENTS_PROGRESS — 客户付款能力（外国人友好支付）施工清单

> 本文件是**自包含施工清单**：任何新会话（含定时续跑 automation）据此继续施工，无需原会话上下文。
> 计划批准于 2026-10-25（分支 `feat/v24-trip-center`）。

背景与目标（一段话）

H5 站点要能向外国客户收款：Visa/Mastercard/Amex/JCB 信用卡 + Alipay + WeChat Pay（Stripe 一个集成全覆盖，hosted Checkout，测试模式免费；网关做成可替换协议，日后可换 Airwallex）。两种收款形态：(a) **pay_now** 立即支付报价金额；(b) **save_method** 保存银行卡（token 化保险库，原始卡号永远不经过我们服务器），公司代客户订票后由运营侧扣款（concierge 代订收款）。与铁律 22 的关系：渠道侧订票仍走 handoff 不变；本功能是"公司↔客户"的收款层，明确豁免并写入设计文档。铁律 23：每条支付记录必须带退改/退款摘要文案。

硬约定（违反即返工）

- 分支 `feat/v24-trip-center`；每完成一个任务**只** `git add` 本任务相关文件后 commit（工作区有大量他人未提交改动，严禁 `git add -A` / `git add .`）。commit message 中文、风格对齐 git log。
- 无新 pip 依赖：Stripe 用 httpx 手写 REST 调用 + 手写 HMAC-SHA256 webhook 验签。
- 金额约定：**yuan float**（repo 惯例），只在 Stripe adapter 内部转 cents。
- 测试门槛：`.venv/Scripts/python -m pytest --cov=app --cov-fail-under=75` 绿；`ruff format app tests && ruff check app tests` 干净。
- 禁止把 key 写进代码/测试；一切走 config（`.env`）。
- 并发续跑防撞规则（lease）：开工前若清单里有任务标 `[IN PROGRESS lease: <ISO时间>]`，且该时间距今 < 40 分钟 ⇒ 本次运行**只退出不施工**（原会话可能正在干）；≥ 40 分钟才可接管（改写 lease 时间）。领任务时立刻写 lease，完成后改 `[x]` 并删 lease 行。
- dev server 用 `D:\chinatrip\dev_server.cmd` / `scripts/dev_watch.py watch`（禁 uvicorn --reload）。

清单

- [x] T0a 本进度文件建立（`DESIGN/PAYMENTS_PROGRESS.md`）
- [x] T0b 每 20 分钟续跑 automation 建立（CronCreate；全部任务完成后它会自删）
- [IN PROGRESS lease: 2026-10-25T10:14:00+08:00] T1 配置键（`CHINATRIP CODES/app/config.py`）
  在 booking_auto provider 键块后加：
  `payments_backend: str = "mock"`（"mock"|"stripe"）、`stripe_api_key: str = ""`、
  `stripe_webhook_secret: str = ""`、`payments_default_currency: str = "cny"`、
  `payments_success_url: str = ""`、`payments_cancel_url: str = ""`（空=从 request.base_url 推导）、
  `payments_operator_token: str = ""`（空=运营扣款端点整体 403 禁用）、
  `payments_db_url: str = "sqlite:///data/payments.db"`。
  验收：ruff 干净；旧测试不回归。
- [ ] T2 服务层（`CHINATRIP CODES/app/services/payments.py`，新文件）
  - `PaymentRecord` dataclass/pydantic：id, device_id, username|None, kind("pay_now"|"save_method"), amount_yuan|None, currency, description, refund_summary, status("pending"→"succeeded"/"canceled"/"refunded"；save_method 成功→"method_saved"), provider, provider_ref, customer_ref, saved_method_label(品牌+末4位), handoff_id|None, trip_item_id|None, created_at/updated_at/paid_at。
  - `PaymentStore`：内存 dict + SQLite write-through，**照抄 `app/services/state_persist.py` 的 SqliteStateBackend/PersistentTripItemService 模式**（load-all on boot、逐变更 upsert、坏行 warning 跳过、单连接+threading.Lock+check_same_thread=False）。自建文件库（settings.payments_db_url）。
  - `PaymentGateway` 协议 + `MockGateway`（返回 `…/h5/?mockpay=<payment_id>` 假收银台 URL，零网络）+ `StripeGateway`（httpx POST form-encoded 到 `https://api.stripe.com/v1/checkout/sessions`：pay_now 用 `mode=payment` + `line_items[0][price_data]`（unit_amount=cents, currency），save_method 用 `mode=setup` + 先建 customer；`success_url/cancel_url`；Bearer sk_ 认证）。
  - `verify_stripe_signature(payload_bytes, sig_header, secret, tolerance=300s)`：按 Stripe t= 与 v1= 方案手写 HMAC-SHA256 常时比较；失败抛 `PaymentError`。
  - `get_payment_service()` 进程级单例（照 trip_items.py 的 lazy singleton 依赖模式）。
  验收：ruff 干净。

- [ ] T3 API 层（`CHINATRIP CODES/app/api/payments.py` 新文件 + `app/main.py` 注册）
  - `POST /api/v1/payments/checkout`（X-Device-ID；body: kind, amount_yuan?, currency?, description?, refund_summary?, handoff_id?, trip_item_id?, email?）→ `{payment_id, checkout_url, status:"pending"}`；pay_now 必须有 amount_yuan>0。
  - `GET /payments/{id}`、`GET /payments/mine?device_id via header`。
  - `POST /payments/{id}/confirm`：**仅 mock 后端**（settings.payments_backend != "mock" ⇒ 403 operator_only），模拟 webhook 成功（SMTP dev-echo 同款豁免逻辑）。
  - `POST /payments/webhook/stripe`：无设备鉴权；Stripe-Signature 验签 → 按事件类型（checkout.session.completed / setup_intent.succeeded / charge.refunded）更新 store。
  - `POST /payments/{id}/charge`：运营侧对已存卡扣款（body: amount_yuan, description）；要求 header `X-Operator-Token` == settings.payments_operator_token（未配置 ⇒ 403）。
    验收：`python -c "from app.main import create_app; create_app()"` 可启动；ruff 干净。
- [ ] T4 模型与迁移（`app/models.py` + `CHINATRIP CODES/alembic/versions/0012_payments.py`）
  - models.py 加 `Payment` ORM（表 `payments`，列对齐 PaymentRecord；JSONB refund payload 可简化为文本列）。
  - alembic 照 `0010_reservation_fields.py` 手写 `op.execute` 风格：revision `w12_0012`，down_revision `w12_0011`，upgrade 建表 + downgrade DROP TABLE IF EXISTS。
    验收：`.venv/Scripts/python -m alembic upgrade head` 本地跑通（PG 可用时）或至少 alembic 能加载链。
- [ ] T5 `.env.example` / `.env` 付款键块（注释对齐现有风格；`.env` 只加空键不填值）
- [ ] T6 H5 支付卡（`CHINATRIP CODES/app/static/index.html` + `app/static/i18n.js`）
  - 预订 tab 内新增「💳 支付 / 代订收款」卡：pay-now 表单（金额、币种 cny/usd/eur、描述、可选关联 handoff_id/trip_item_id）+ save-card 表单（email）→ 打开 checkout_url → 返回后轮询 `GET /payments/{id}` 展示结果（pending/succeeded/refunded/method_saved 徽章，双语）。
  - 「我的付款」列表（api() 同款鉴权头）。
  - booking-auto L3 order-intent 成功后加「创建代订收款」按钮（预填 handoff_id）。
  - i18n.js 补 `pay.*` 键（中英）。mock 后端显示提示横幅（dev 专用 confirm 按钮说明）。
    验收：`http://127.0.0.1:8000/h5/` 手测 mock 全流程零网络可用；无 console 报错。

- [ ] T7 测试（`CHINATRIP CODES/tests/test_payments.py` 新文件）
  - store 往返 + 重启重载；mock checkout→confirm→succeeded；
    stripe 后端下 `/confirm` 403；webhook 坏签名 400 + 正确 HMAC 签名 200；
    StripeGateway create-session form payload（monkeypatch httpx，仿 amap 测试风格）；
    `/charge` 未配 token 403 / 配了错 token 403。
  - 全套 pytest 覆盖率 ≥75% 保持；ruff 干净。
- [ ] T8 设计文档纪律（`DESIGN/`）
  - 先读当前版本头（并行会话可能已推进版本）。给现行设计 md 追加「支付能力（客户代收层）」
    小节：定位=公司↔客户收款（铁律 22 渠道 handoff 不变 + 豁免说明）、双形态、Stripe 网关可换、
    铁律 23 摘要先行。`INDEX.md` 决策速查加一行。若按纪律需版本升级（v2.4→v2.5）则按硬规则
    三件套同步（md + svg 标题 + INDEX 当前版本）并存档旧对到 `DESIGN/ARCHIVE`（两侧 hash 一致，
    AutoClaw 镜像不存在则跳过并注明）。
- [ ] T9 终验
  - pytest 全绿（覆盖率门槛内）、ruff 干净、H5 手测记录写回本文件「验收记录」节。
  - 全部勾完 ⇒ 删除续跑 automation（CronDelete）并停。

验收记录

- （每完成一任务在此追加一行：日期、执行者、关键结果/数据）